CCTV-지상드론(UGV) 연계 산림 감시·대응 시스템.

팀명: Clue

팀 소개

이창호

팀장 /
Gateway & VMS Server

지상드론(UGV) 연계 중계기 및
무선 통신 시스템 개발

김가영

VMS Client

Qt VMS 통합 클라이언트 애플리케이션 구현

김태훈

영상처리

지상드론(UGV) 영상 처리,
CCTV 이벤트 감지 및
바운딩박스

한정수

UGV BSP & Application

UGV 하드웨어 제어 및
센서 인터페이스 최적화 개발

목차

01 프로젝트 개요

02 전체 시스템 아키텍처

03 시연 영상

04 지상드론(UGV)

05 게이트웨이 / VMS 서버

06 영상처리

07 VMS 클라이언트

08 마무리

개발 배경 및 목적

기존 문제



"꺼져 있던 눈"- 산불감시 CCTV, 최근 5년 12건 '실시간 감시 실패'

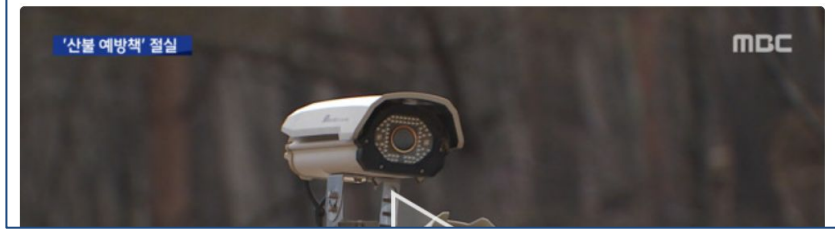
한국에너지안전신문 업데이트 2025.10.13 08:53 | 댓글 0

[이슈] 여의도 44배 면적 카메라 1대가 감시… 말로만 '영상관제'

가

38만 헥타르 국유림에‥산불 감시 CCTV는 고작 31대

입력 2022-03-16 20:16 | 수정 2022-03-16 20:22



SOLUTION

CCTV 1차 감시 + UGV 2차 현장 대응

상황 대응의 정확성 향상

CCTV 광역 감시망과 UGV의 근접 확인을 결합한 입체적 관제 체계 마련

선조치 및 신속한 의사결정 지원

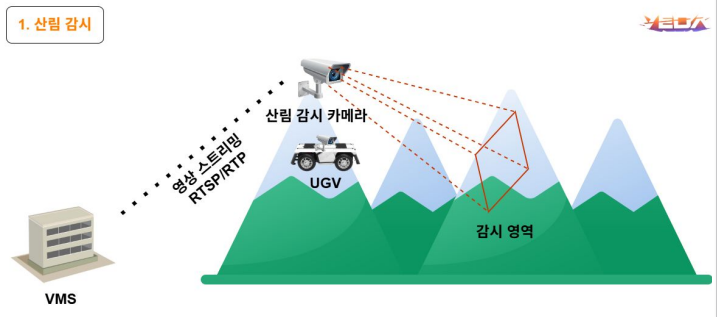
인력 투입 전 UGV 현장 사전 파악으로 안전 확보 및 골든타임 사수

실시간 상황 인지 체계 구축

사람이 접근하기 험난한 지형에서도 단절 없는 현장 스트리밍 제공

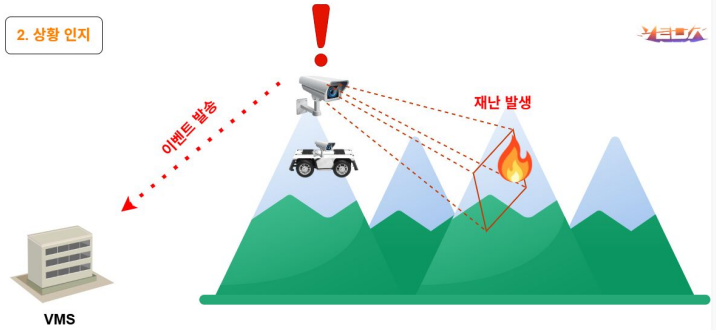
핵심 기능 및 기대효과

산림 감시



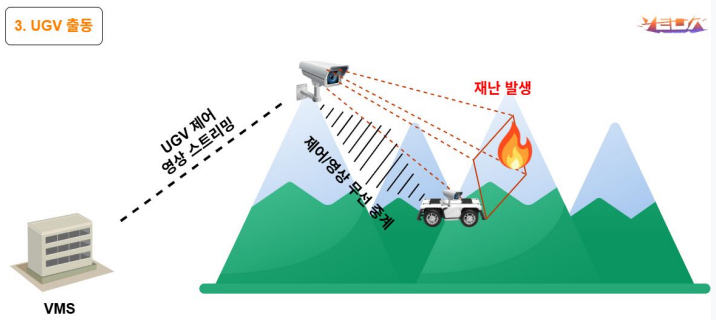
고정형 CCTV를 통한 광역 산림 상시 모니터링 및 실시간 영상 전송 (RTSP/RTP)

상황 인지



이상 징후(재난 발생) 포착 시 VMS로 즉각적인 이벤트 알림 발송 및 관제자 상황 인지

UGV 출동



사각지대 근접 확인을 위한 UGV 현장 투입 및 중계기를 통한 원격 제어/영상 스트리밍

---

1 감시 사각지대 해소

UGV의 기동성으로 고정 CCTV의 사각지대 보완, 험준한 산림에서도 틈새 없는 감시망 구축

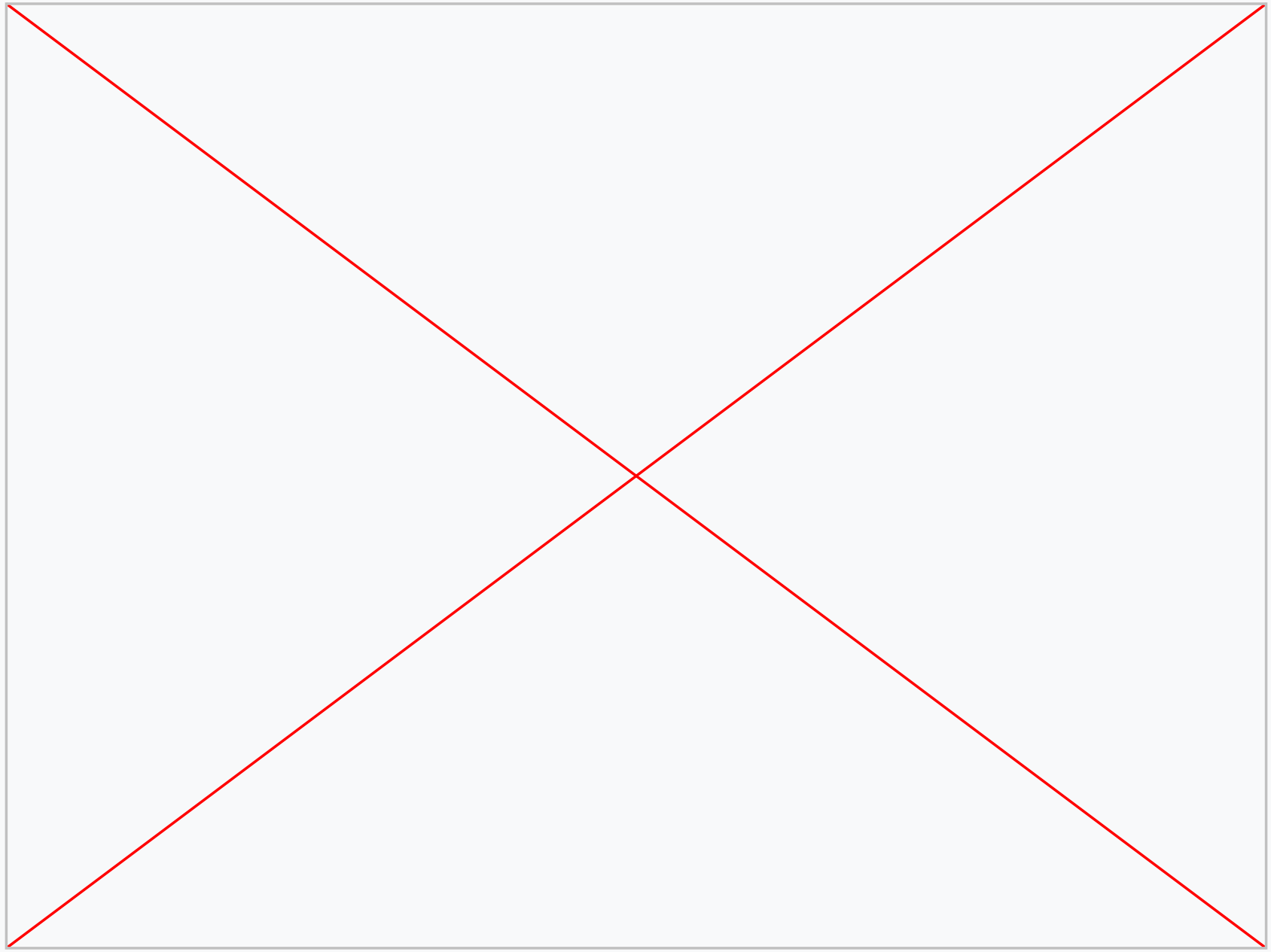
2 운영 효율성 및 비용 절감

소수 인력으로 다채널 스트리밍과 원격 제어를 통해 광범위한 산림 관제 가능

3 인력 안전 확보

인력 직접 투입 전 UGV로 현장 사전 파악, 위험 지형에서의 골든타임 내 의사결정 지원

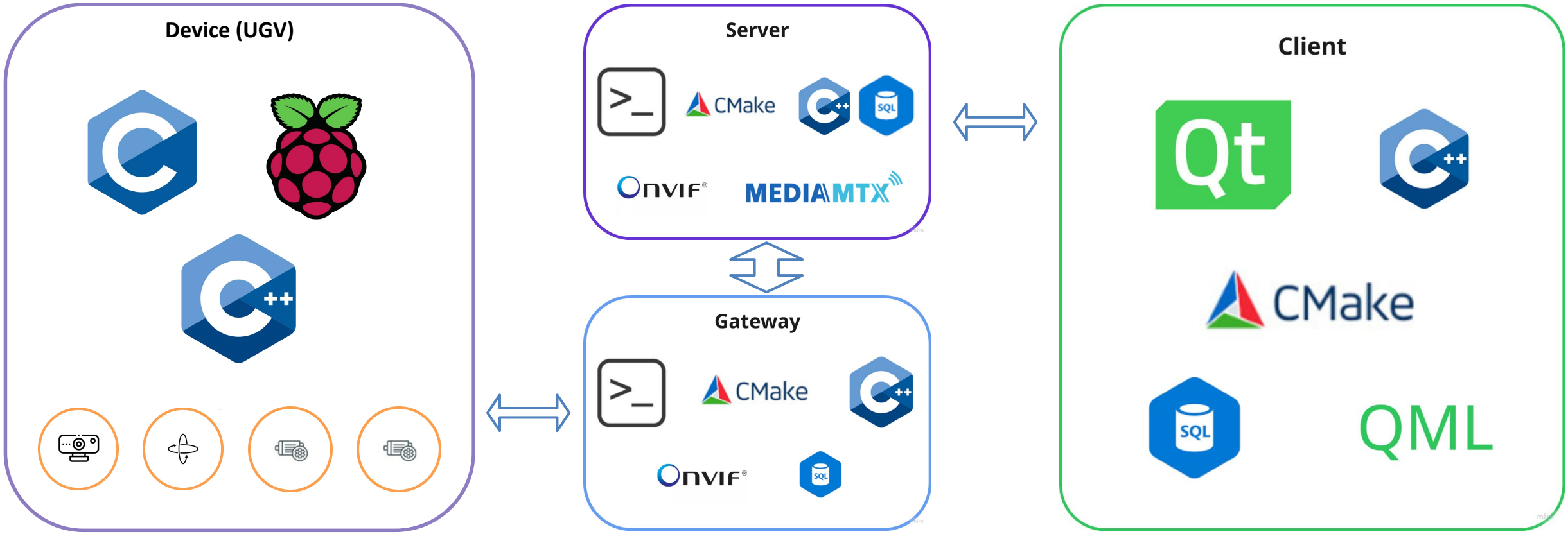
시연 영상



개발 프로세스

Git 브랜치 운용 및 효율적인 자료 공유 프로세스 구체화

기술 스택



OpenCV

gstreamer

SUNAPI

Websocket

docker

Jira Confluence GitHub Google Drive Slack

협업 전략

VEDA 3 Clue

Korea, South

Jira

드라이브

Repositories

Find a repository... Type

UGV_APP Private
UGV_APP 개발용 레포 입니다.
C ☆ 0 MIT 0 0 0 Updated 16 hours ago

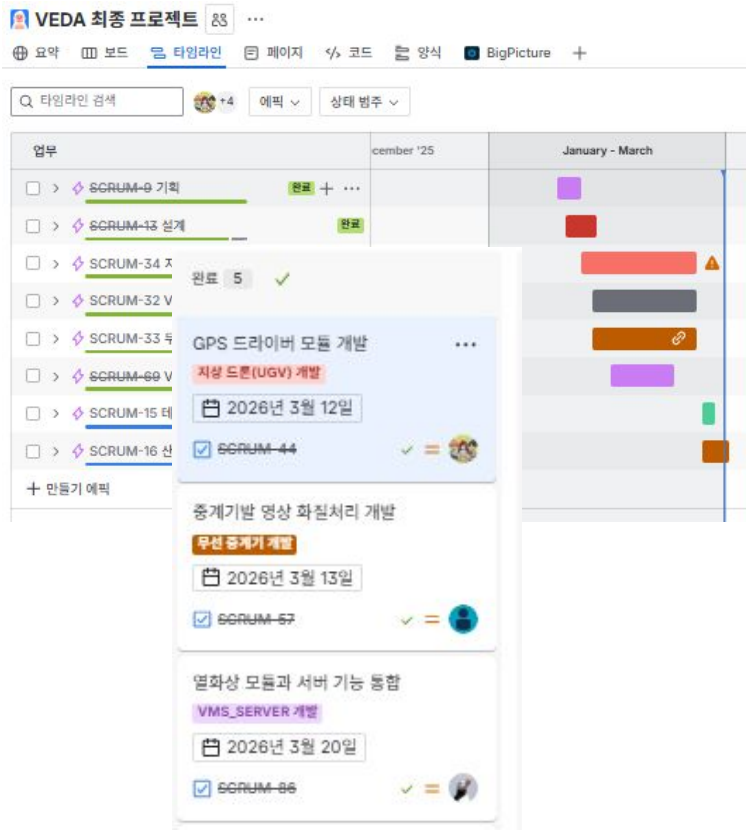
nRF24_Module Public
Forked from aarossig/nrfnet
Using NRF24L01 as a virtual network device under Linux.
C++ ☆ 0 Apache-2.0 21 0 0 Updated 19 hours ago

VMS_Server Private
VMS 서버 입니다.
C++ ☆ 0 MIT 0 0 0 Updated 20 hours ago

Gateway Private
중계기 개발 깃입니다.
C++ ☆ 0 MIT 0 0 0 Updated 20 hours ago

VMS_QT Private
VMS_QT 인터페이스 입니다.
C++ ☆ 0 0 0 0 Updated 5 days ago

Docker-env Private
Docker 공통 개발 환경을 관리합니다.
Shell ☆ 0 MIT 0 0 0 Updated last week





통합 개발 환경

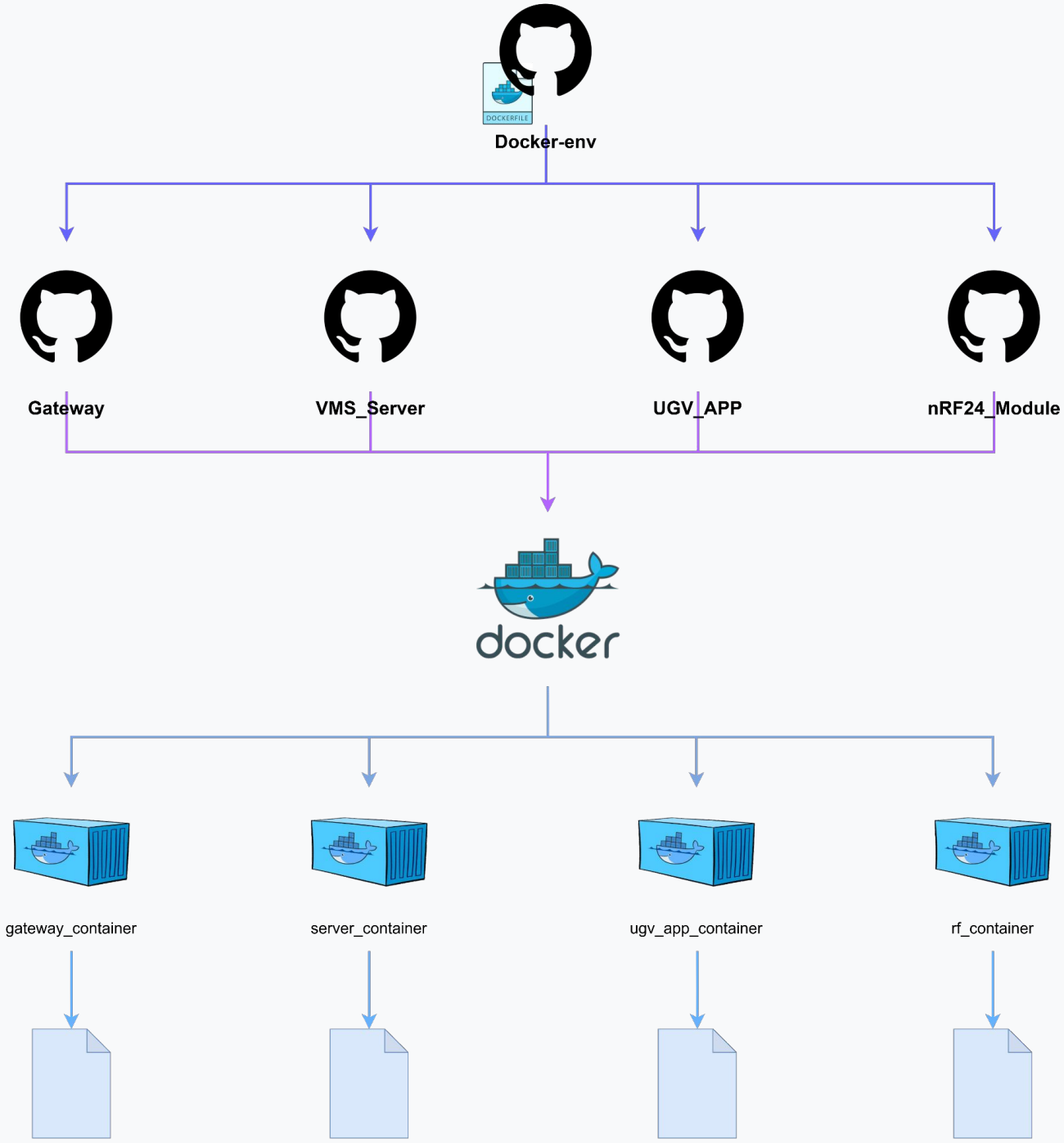
공통 환경 통합 및 배포

공통 환경 기반: Qt를 제외한 서버, 중계기, UGV 등 모든 영역에 Linux aarch64 기반 환경 적용

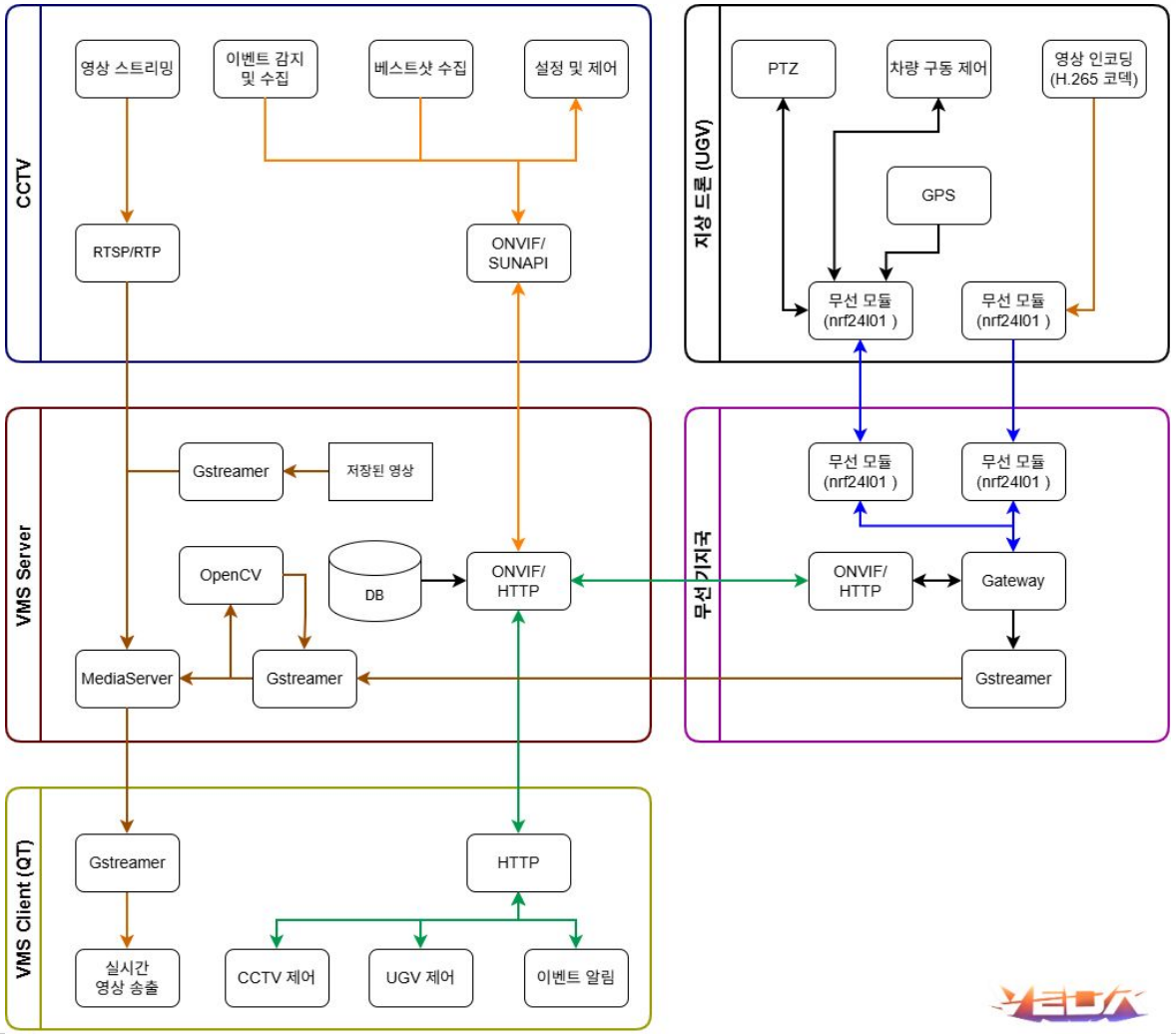
중앙 집중형 관리: Docker-env Git 레포지토리에서 Dockerfile 및 빌드 스크립트 통합 관리

서브모듈 연동: 각 모듈이 Docker-env를 서브모듈로 포함

효율성 극대화: 멤버들은 해당 서브모듈에서 최신 Docker를 Pull하여 즉시 사용 가능



전체 시스템 아키텍처


CCTV
영상 스트리밍
이벤트 감지 및 수집
베스트샷 수집
설정 및 제어
RTSP/RTP
ONVIF/ SUNAPI
지상 드론 (UGV)
PTZ
차량 구동 제어
영상 인코딩 (H.265 코덱)
GPS
무선 모듈 (nrf24l01)
무선 모듈 (nrf24l01)
VMS Server
Gstreamer
저장된 영상
OpenCV
DB
ONVIF/ HTTP
MediaServer
Gstreamer
무선 기지국
무선 모듈 (nrf24l01)
무선 모듈 (nrf24l01)
ONVIF/ HTTP
Gateway
Gstreamer
VMS Client (QT)
Gstreamer
HTTP
실시간 영상 송출
CCTV 제어
UGV 제어
이벤트 알림

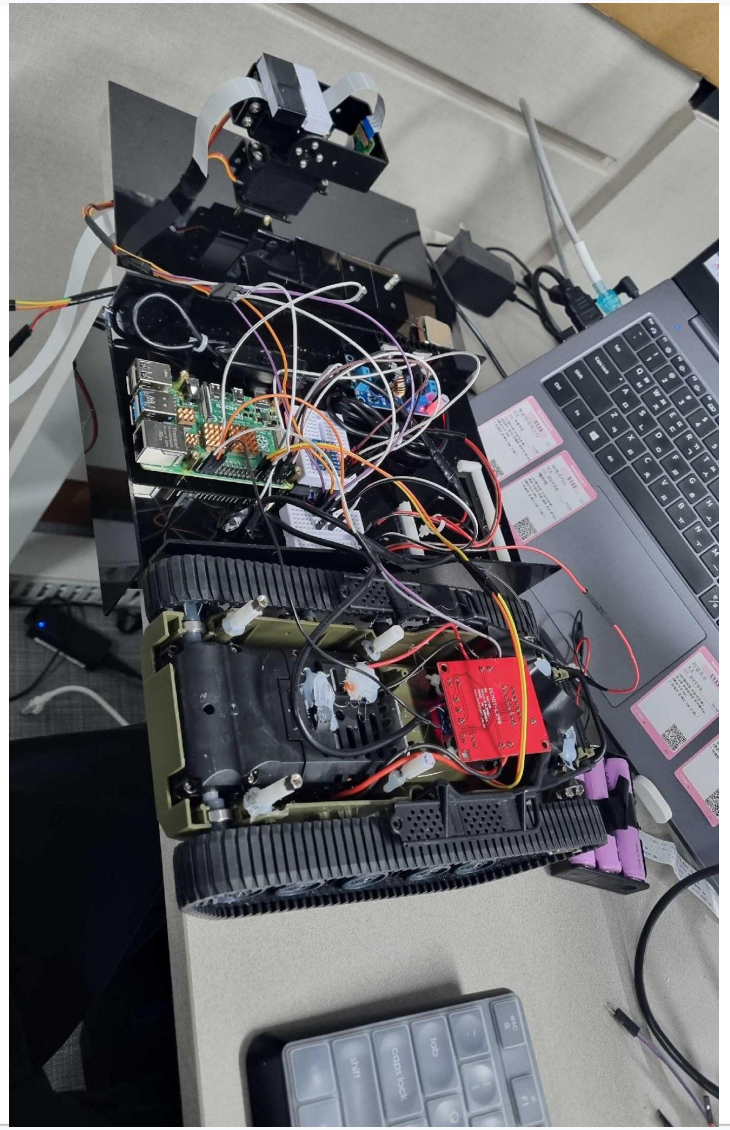
지상드론

(UGV)

한정수 | UGV BSP/Application



UGV: BSP (구현)



Device Driver

- **mg996r_ko**
  Pan/Tilt 모듈
- **l298n_ko(motor)**
  차량 motor 모듈
- **mpu6050_ko**
  Gyro 모듈
- **neo6m_ko**
  GPS 모듈

UGV: BSP

- 실시간성
- 일정한 각도 유지



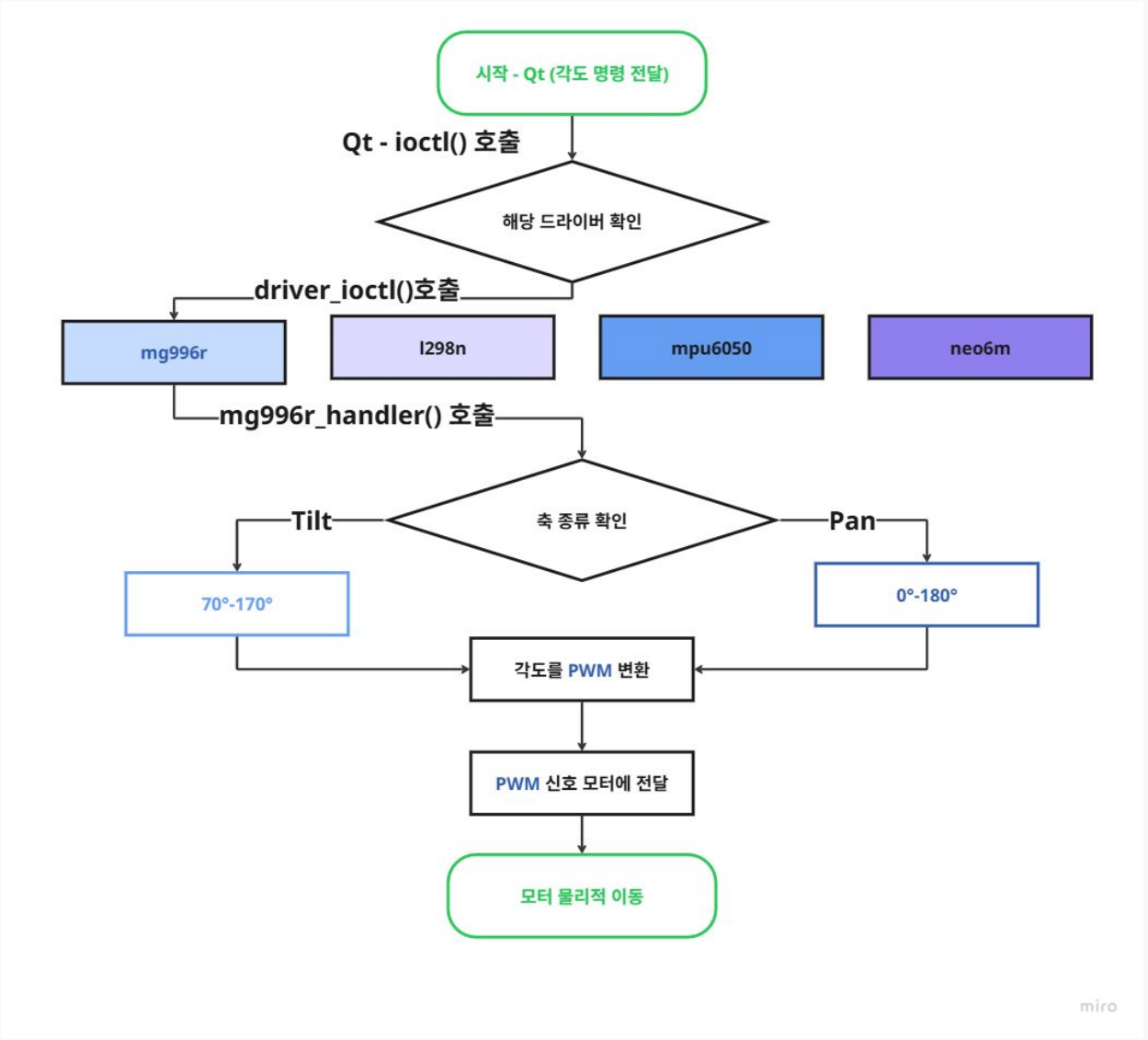
- 가상 리밋
- 초기 위치 값 복귀



UGV: BSP



flow chat



UGV: BSP mg996r — Pan/Tilt 서보 모터

설계 목표

사용자 편의성

실시간 반응, 지연 최소화

일정한 각도 조절

위치 개선 보장

규격 준수

H사 ES 규격서 일부 참고

Motor 보호

Physical 100% 사용 지양

각도 스펙

Pan (좌우)

범위: 0° ~ 180°

리밋 : 미적용

초기 위치 값 복귀 로직 포함

Tilt (상하)

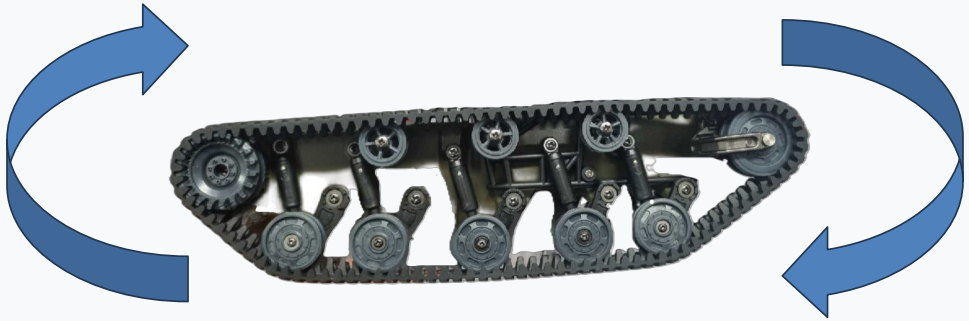
범위: 0° ~ 180°

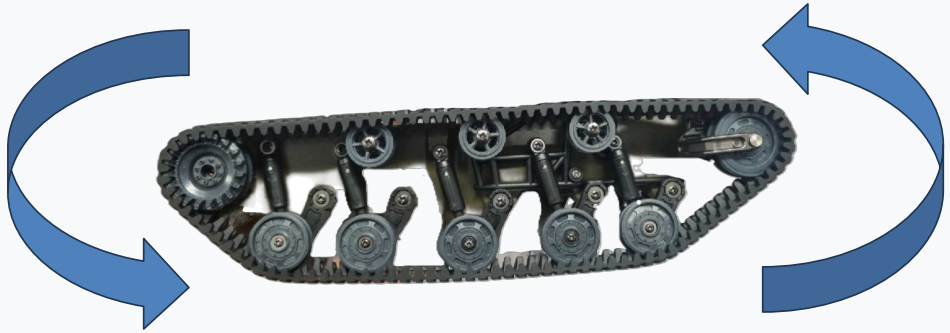
리밋: 가상 리밋 70°~170°

초기 위치 값 복귀 로직 포함

⚠ Tilt에 가상 리밋(70~170°) + 초기 위치 복귀 로직 → 지속 사용 시에도 개선된 위치 보장

UGV: BSP







UGV: BSP l298n — 차량 구동 모터

설계 고려사항

- 모터 전압 8V / 12V 환경 지원
- 댐퍼 없이 물리적 흔들림 보정 — 탱크 바퀴 채택
- UGV 완성 시 무게 고려, PWM 65% 동작 가능 검증
- 0% / 70% / 85% / 100% — 정지 · 저속 · 중속 · 고속

통신 구성

UGV Application

모터 명령 패킷 수신

Gateway (GW)

무선 중계 및 UDP 전달

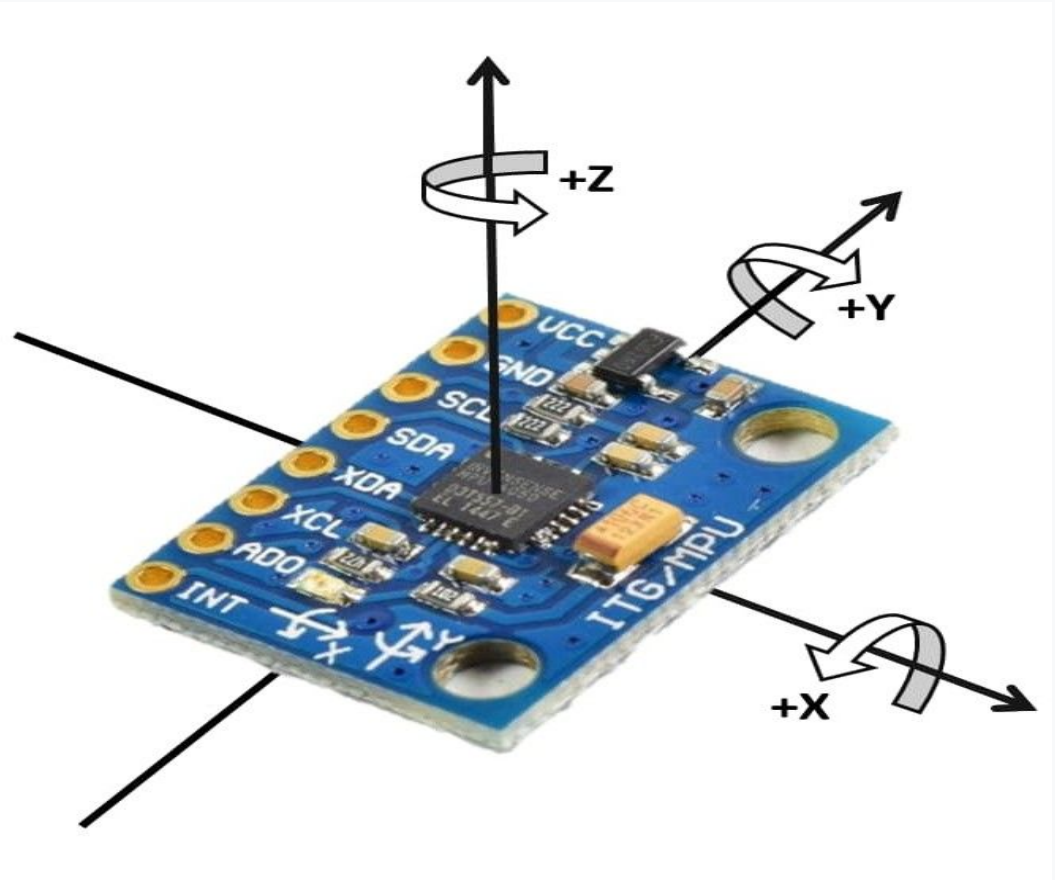
Server

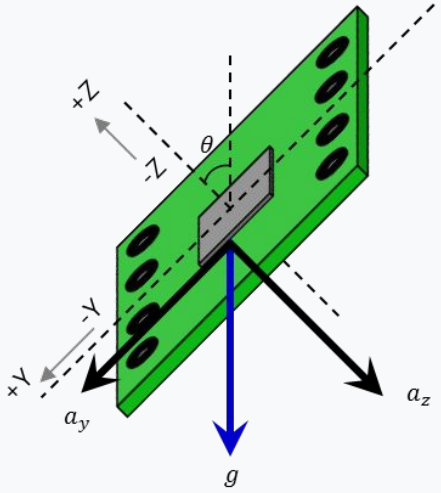
클라언트와 중계기의 통로 역할

3단 속도 구성

모드	PWM	설명
정지	0%	PWM Duty = 0
저속	70%	안전 이동
중속	85%	일반 주행
고속	100%	최대 출력

UGV: BSP






```
[INFO] Gyro calibration done.
```

UGV: BSP mpu6050 — Gyro 센서

스펙 및 설정

최고 스펙

2000°/s (정밀도 저하로 미사용)

채택 스펙

250°/s (정밀도 향상)

측정 데이터

X/Y 각도, X/Y/Z 각속도, 중력 가속도

초기화 로직 및 위치 특정

현재 경사 기준으로 초기값 재설정

영상 안정화 처리 흐름

1. MPU6050 데이터 수신 (I2C)
2. X/Y 각도 계산 (경사 보정 포함)
3. X/Y/Z 각속도 추출
4. 영상 처리팀으로 각도/속도 전달
5. 경사 기반 초기값 자동 초기화

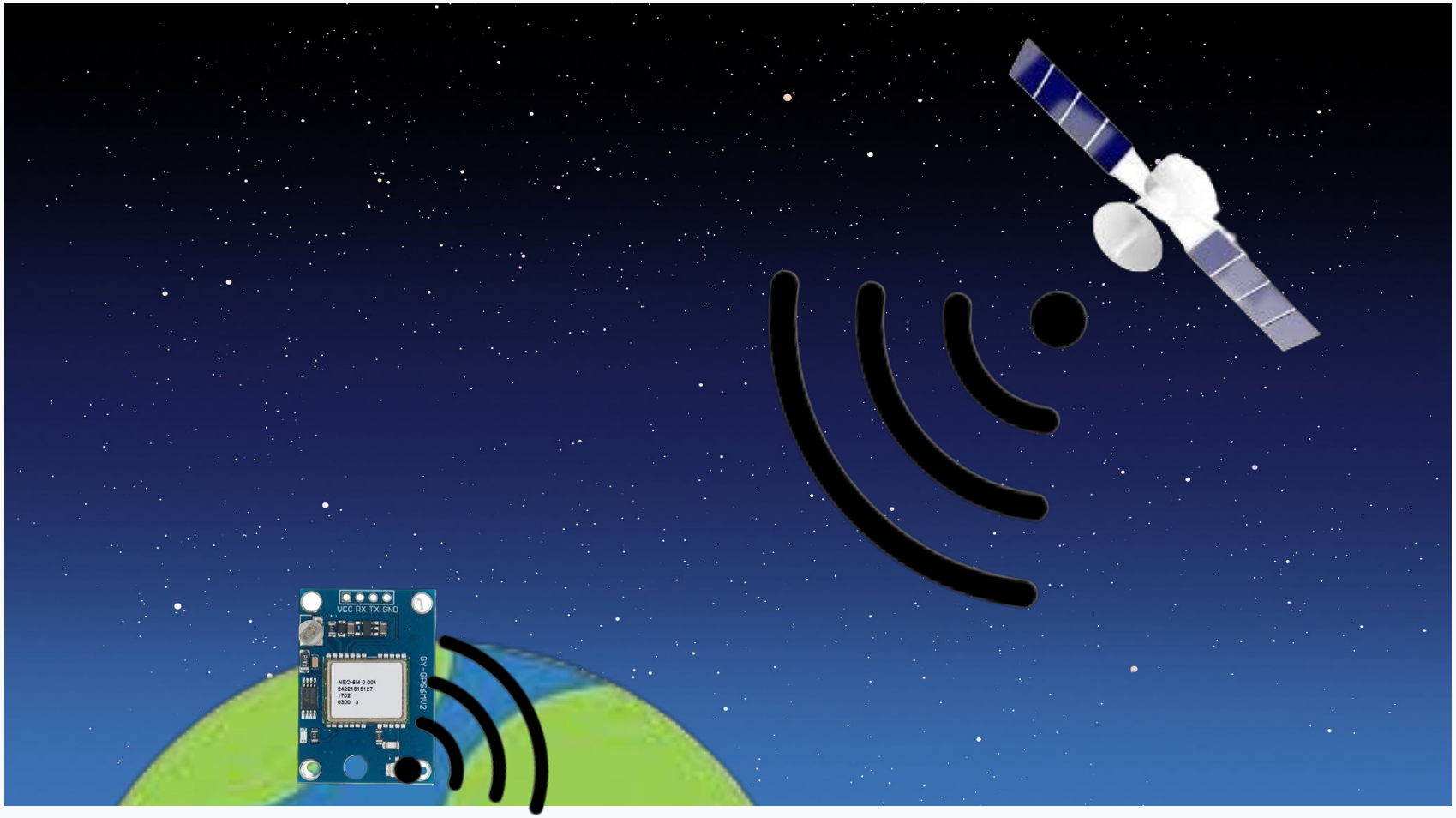
💡 2000°/s 최고 스펙 → 정밀도 저하 확인 → 250°/s 채택으로 영상 안정성 대폭 개선

UGV: BSP

. CEP

. DOP

. 2D RMS



UGV: BSP



flow chat



UGV: BSP neo6m — GPS 모듈

정확도 지표

CEP

원형 오차 확률

Neo6m: 59% 정확도
평균 오차 4~8m (이론 최상 스펙)

HDOP

정도저하율 (DOP)

측정값 3~4
5 이하면 내비게이션 사용 가능 수준

(프로젝트는 2이하의 고정밀도가 필요)

실사용

실제 오차 범위

0~10m 오차
(이론 최고 스펙보다 4M 추가 오차)

위치 처리 로직

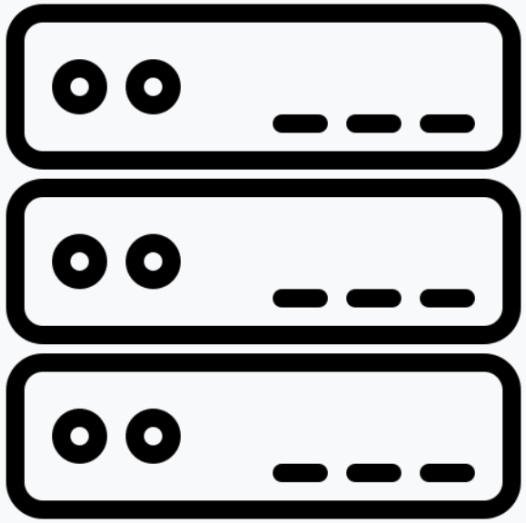
2D RMS
길이 계산법으로 위치 오차 분석

실시간 자체 개선값
Queue 오차 증가 확인 → 실시간 원시값 채택

NMEA 파싱
UART 수신 후 위경도 파싱 및 전송

Gateway / VMS Server

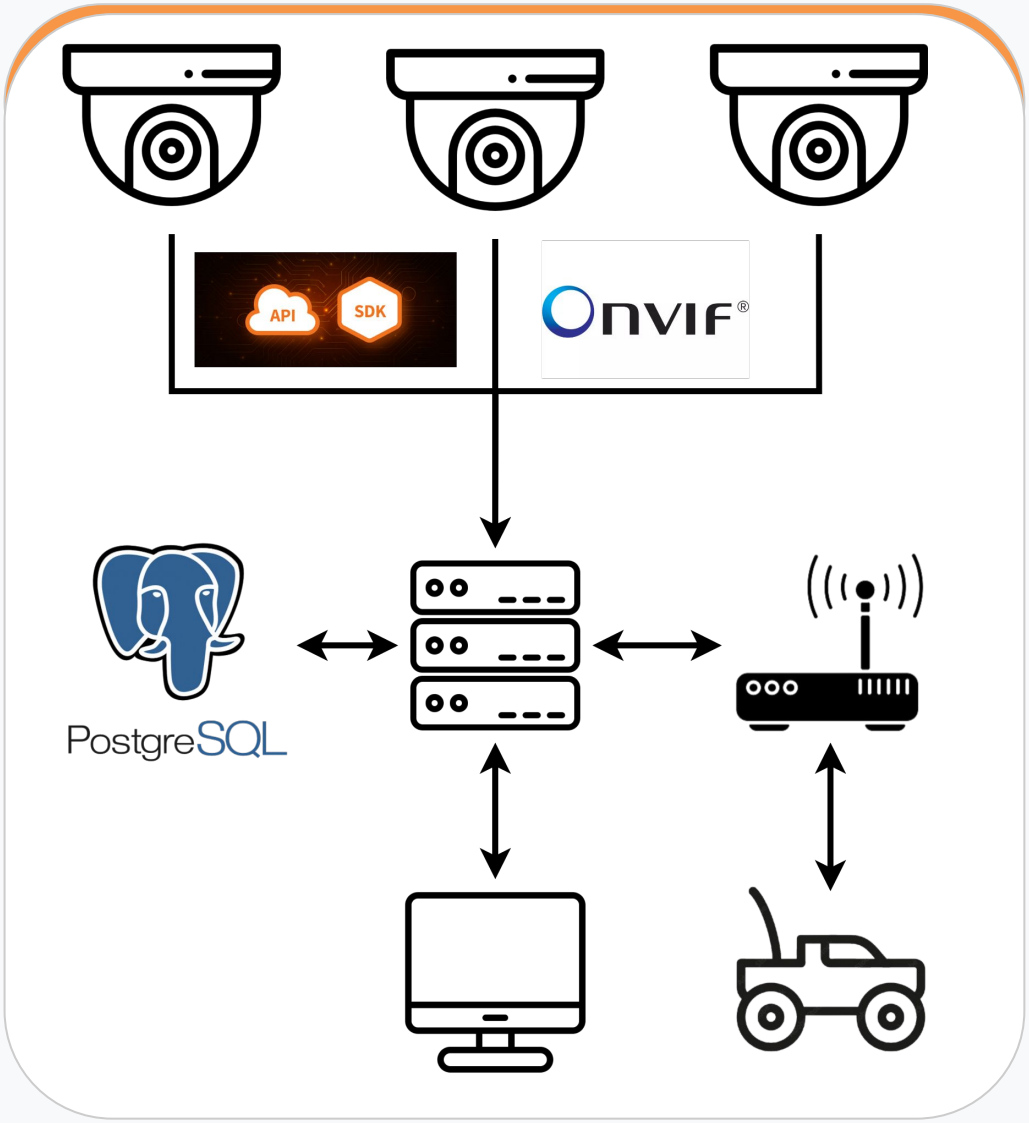
이창호 | 서버/중계기/무선 통신







서버 시스템 개요



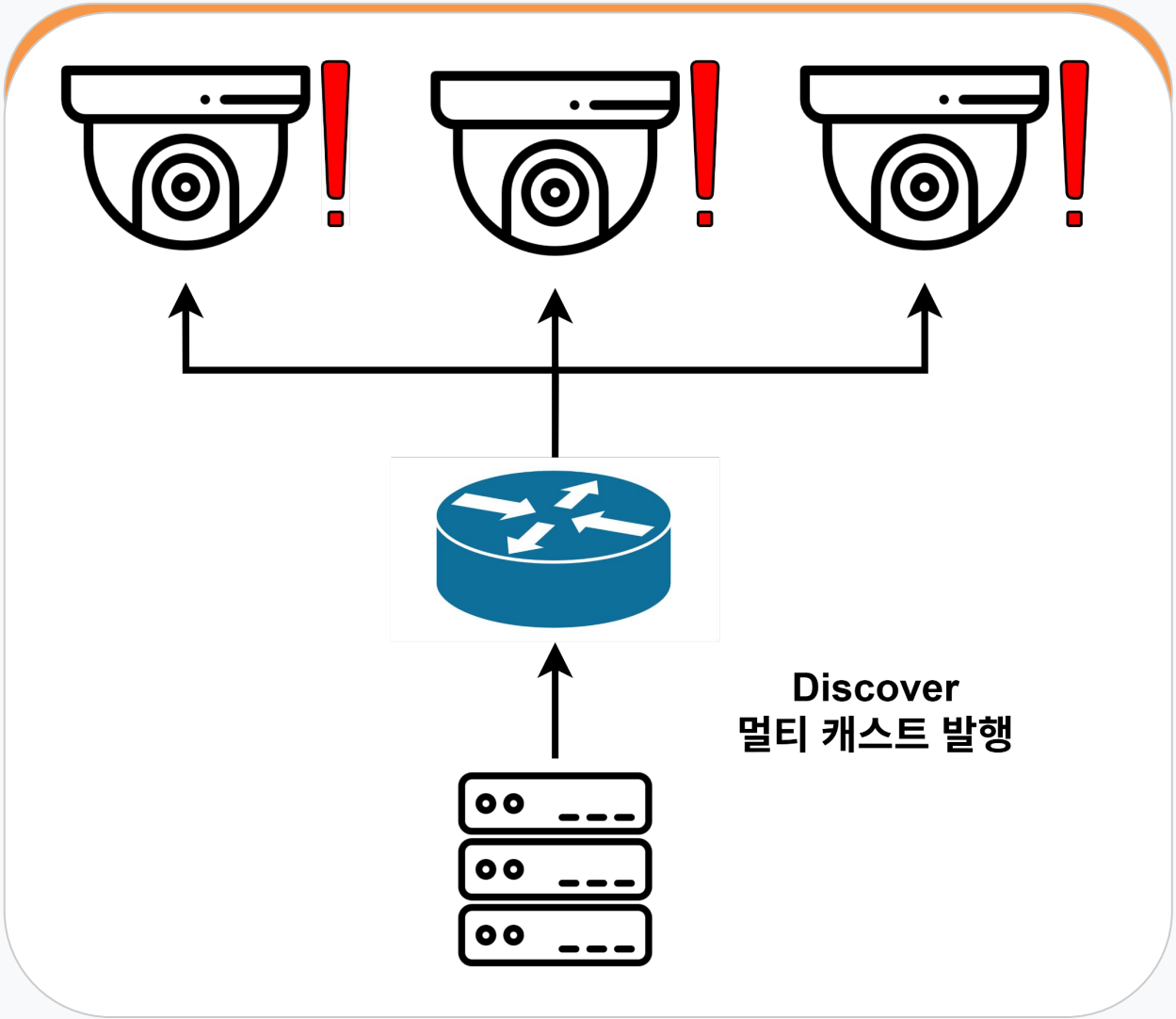
본 시스템의 서버는 영상 기기들과 클라이언트를 연결하는 핵심 브릿지 역할을 수행합니다.

CCTV 및 중계기 서비스: 연결된 디바이스들을 VMS 클라이언트로 안정적으로 서비스

미디어 중계: 저지연, 고품질의 미디어 중계 역할 수행

강력한 보안 세션: JWT(JSON Web Token)를 이용한 무결성 높은 사용자 세션 관리

서버 시스템 (ONVIF + SUNAPI)



ONVIF 표준 활용

Discover: 멀티캐스팅을 통한 망 내 장치 자동 탐색 및 등록

GetCapabilities 등: 모델명 및 속성 구분

CreatePullPointSubscription: 실시간 CCTV 이벤트 수집

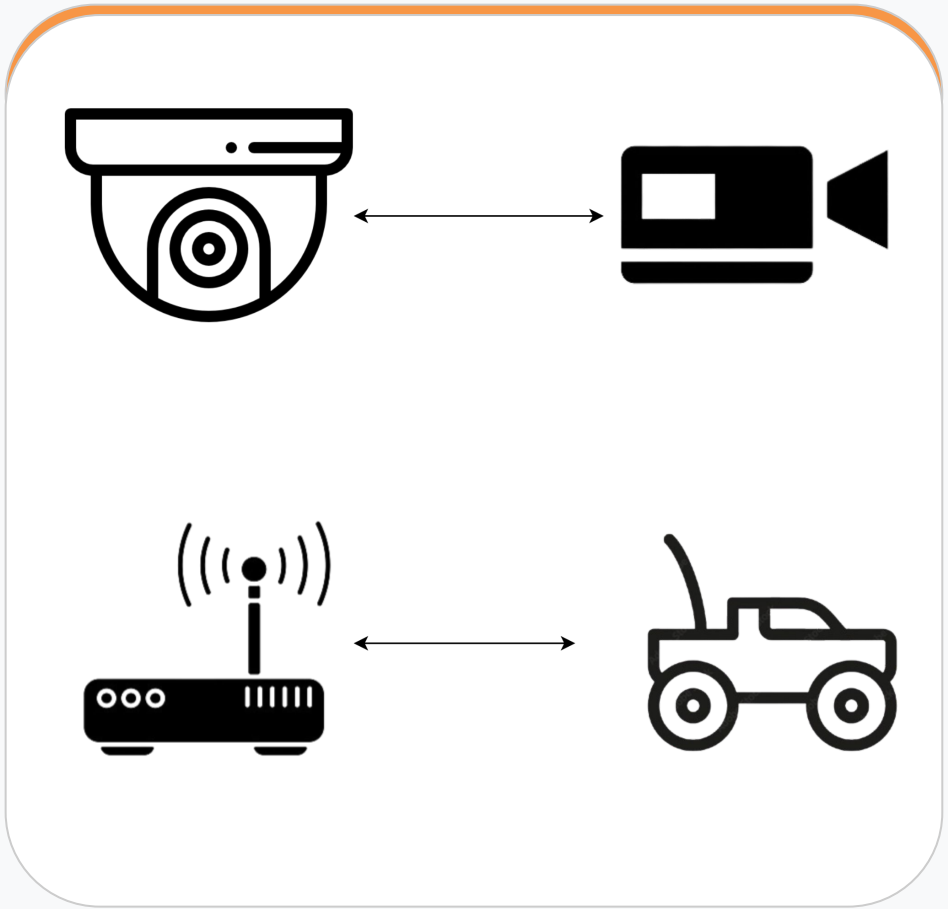
SUNAPI 보완 적용

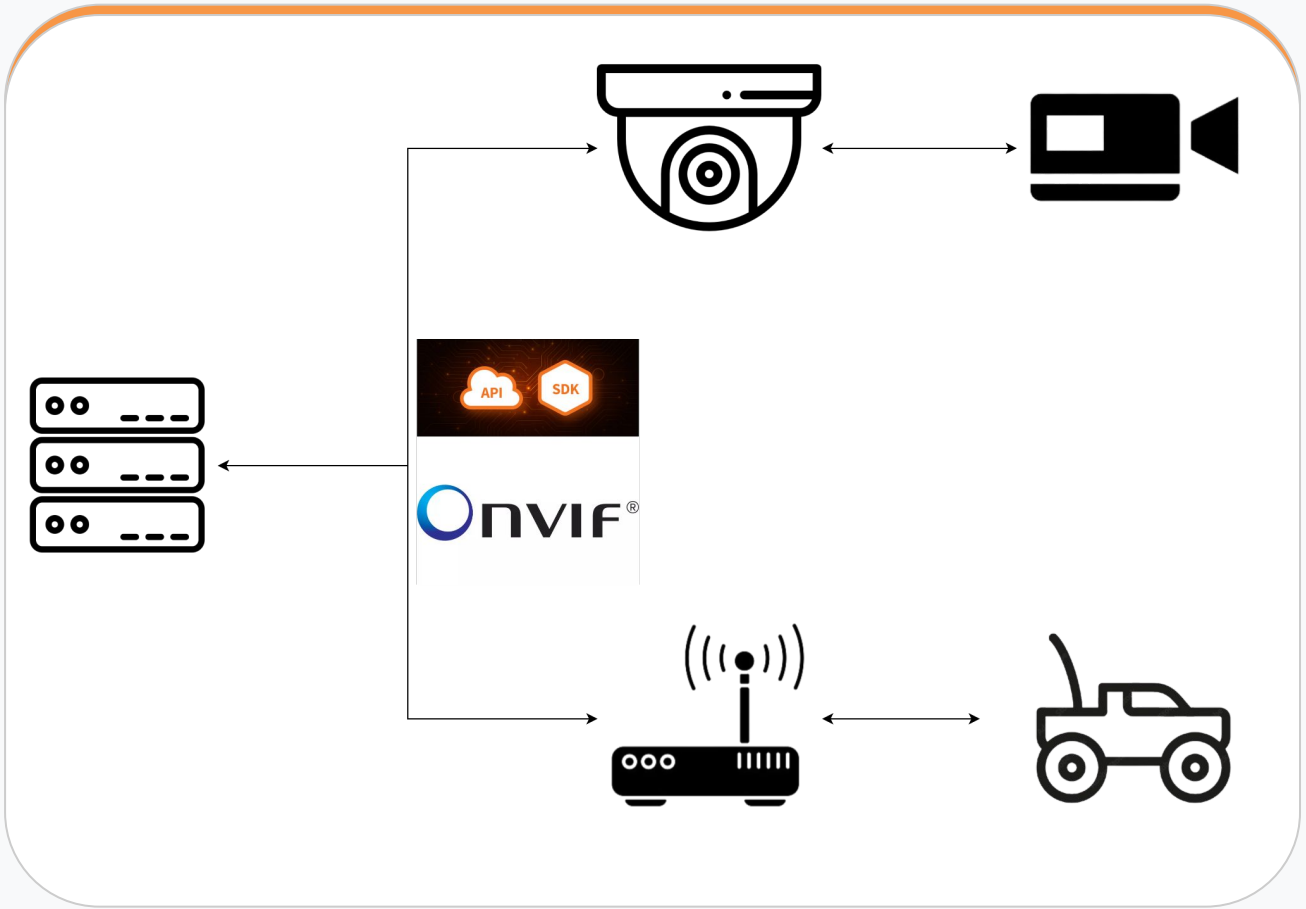
다중 채널 환경에서 모호할 수 있는 RTSP 프로필 안정적 특정

HTTP Digest 인증을 통한 RTSP 주소 획득 예외 처리 적용

중계기 역할 및 무선 제어 프로토콜 설계

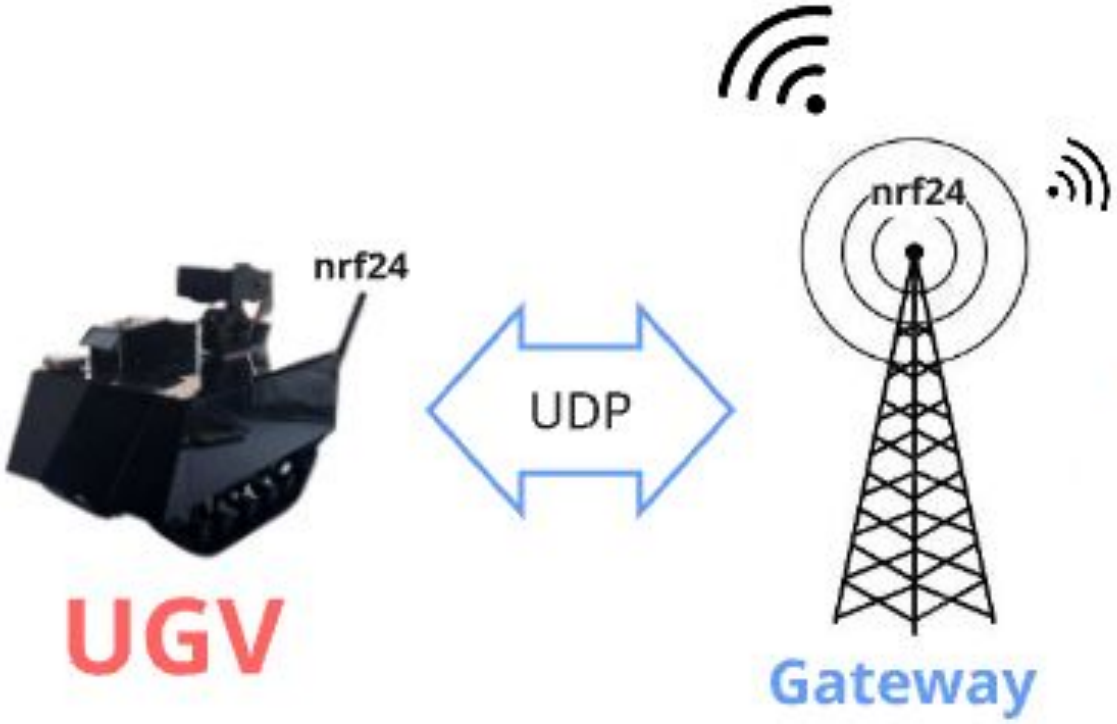
네트워크-무선 통신 사이에서 패킷을 교환하며, 시스템 이식성의 극대화를 위해 ONVIF 표준을 중계기에 도입했습니다.





중복 개발 최소화: 서버가 중계기 및 UGV를 일반 CCTV처럼 인식하도록 구현

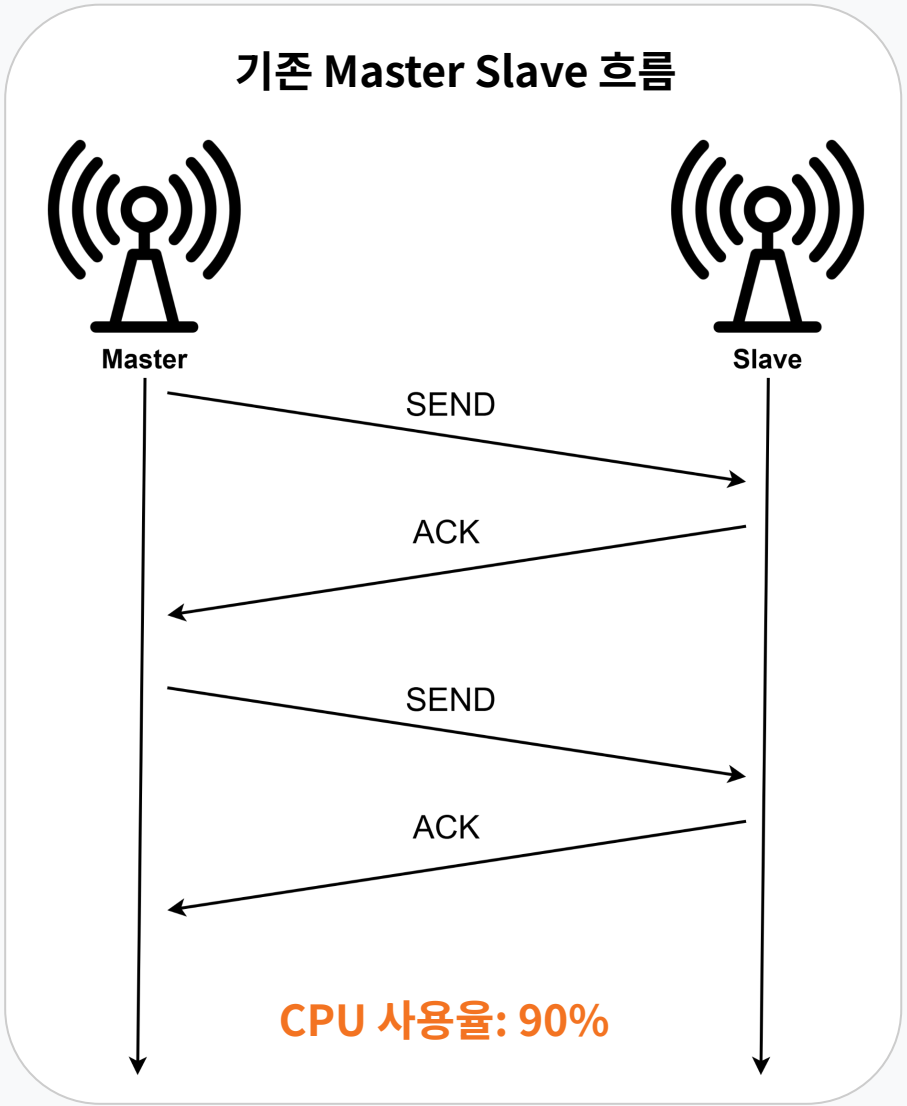
RF 무선 통신 구현 (nrf24)

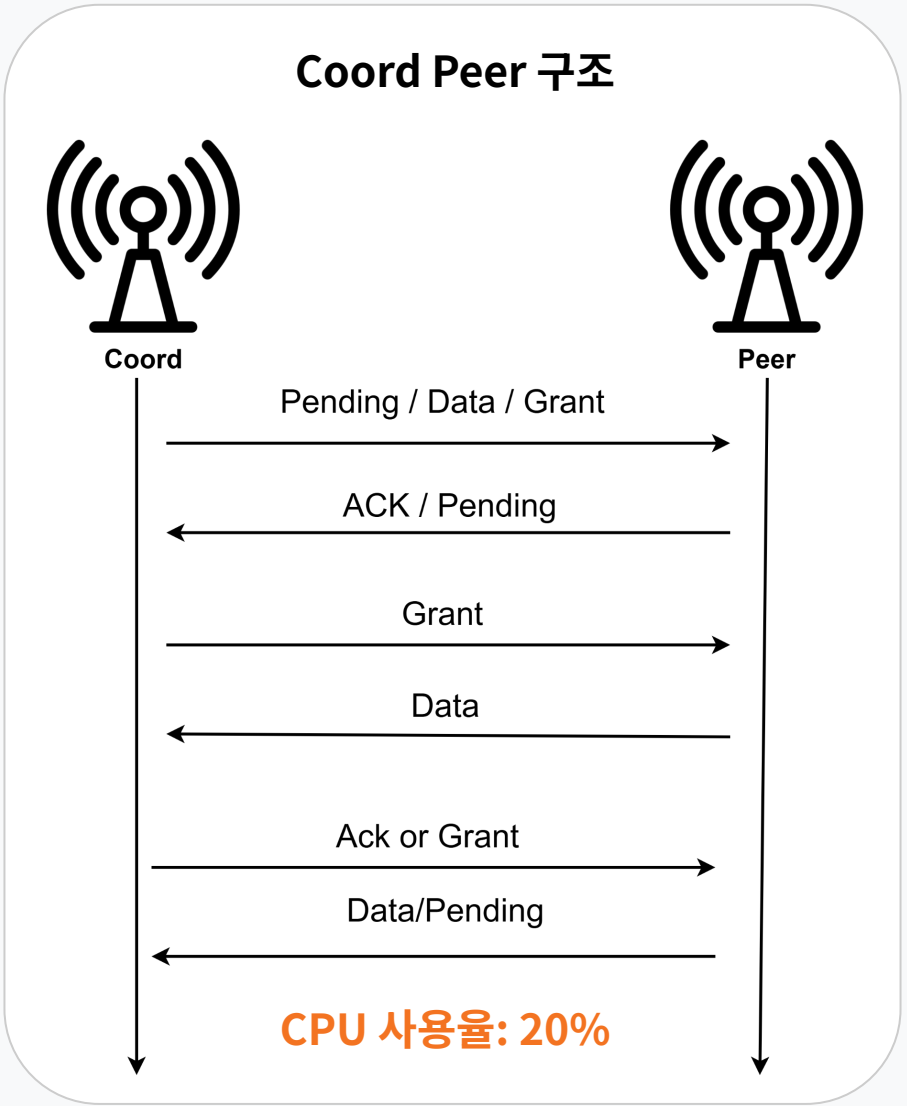


모듈 채택 이유

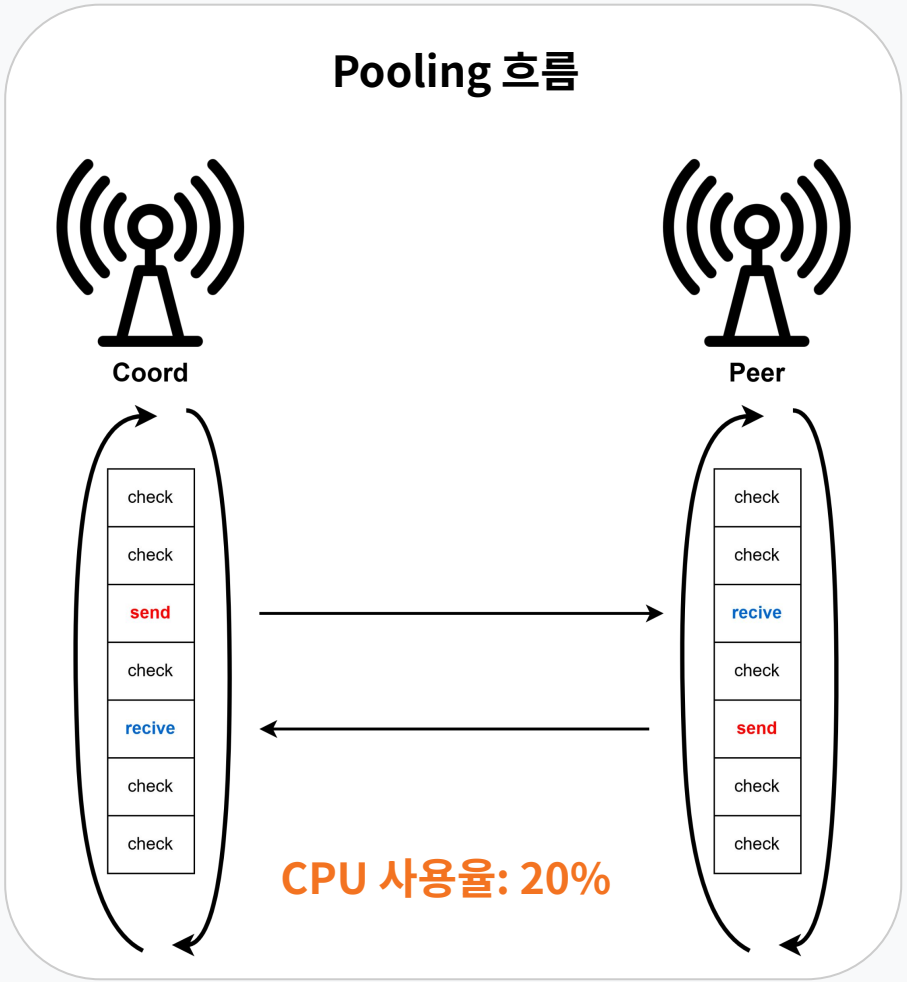
- **WiFi**: 안정적이나 짧은 통신 거리, 기술적 도전성 낮음
- **LTE**: 예산 초과 및 인프라 종속
- **FPV**: 비싼 가격, 전용 카메라 및 전용 영상 송수신기 필요
- **nRF24L01**: 값싼 가격, 비디오 스트리밍 성공 사례 존재
  https://github.com/aarossig/nrfnet

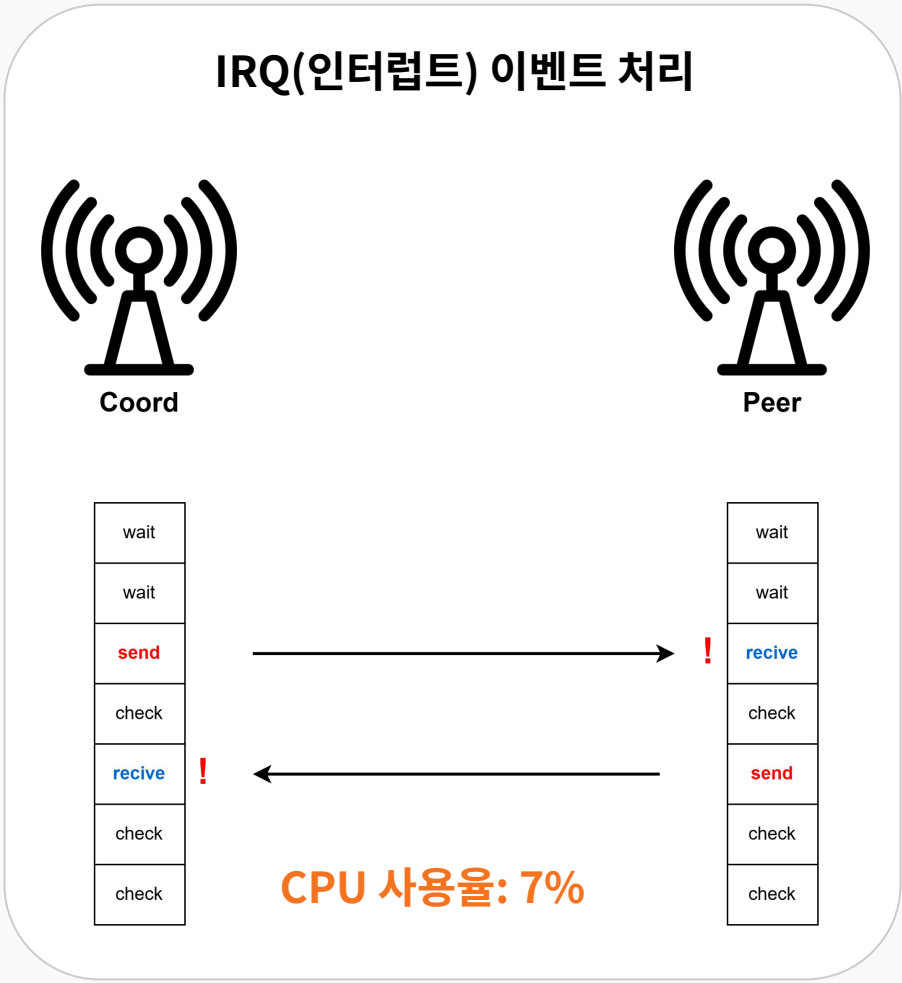
RF 무선 통신 구현 흐름 개선





RF 무선 통신 구현 CPU 최적화

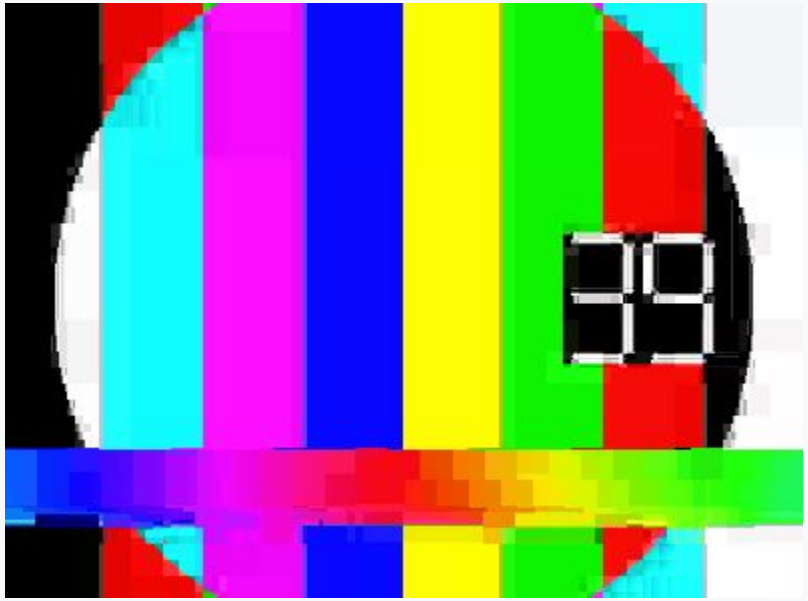




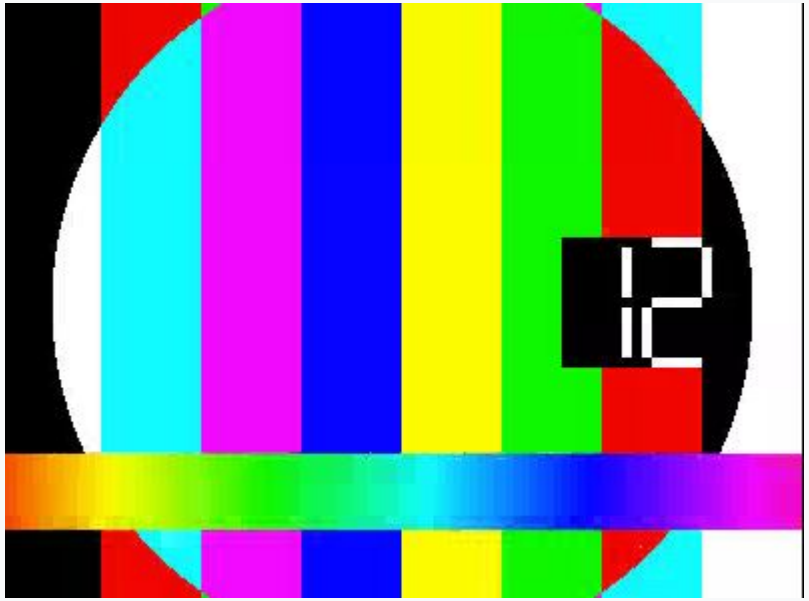

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2307 root      20   0   79760   3368   3148 R  91.7   0.2  12:57.18 ld-linux-aarch6
```

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 1967 root      20   0  153892   3780   3480 S   7.0   0.0   0:11.55 ld-linux-aarch6
```

RF 무선 통신 구현 2채널 개선



1채널 영상 송신
H.264 인코딩 400x300 10fps 50k bitrate



2채널 영상 송신
H.264 인코딩 400x300 10fps 70k bitrate

RF 무선 통신 구현 실시간 한계



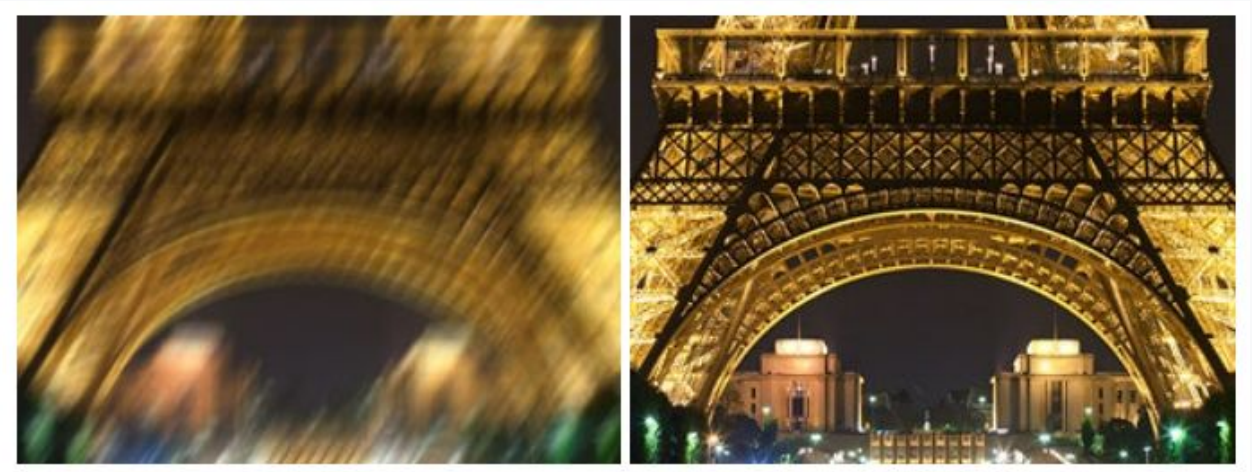
실시간 H.264 128x98 8fps 20kb

실제 송출 한계

- **녹화 영상은 미래 프레임 활용 -> 높은 압축 효율**
- **실시간은 현재 프레임만 사용 -> 동일 화질 더 높은 bitrate**
- **네트워크/버퍼 여유 차이로 손실 영향이 즉시 드러남**

영상처리

김태훈 | 지상드론(UGV) 및 CCTV 영상 처리





비디오 안정화 - 기술 선택

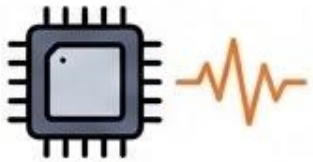
안정화 기법 분류 및 선택 배경



OIS (광학식)

렌즈를 물리적으로
이동하여 보정

❌ **비용·하드웨어**



EIS (전자식)

자이로 센서 의존도 높음,
영상 데이터 활용 한계

❌ **단독 사용 비추천**



DIS (디지털)

순수 소프트웨어로
픽셀 움직임 계산

✅ **베이스라인으로 선택**

DIS의 경제성·연산 효율성과 EIS의 자이로 센서 장점을 결합한
Sensor-Fused DIS (융합형 디지털 안정화) 방식을 독자적으로 구축

비디오 안정화 - 알고리즘 구조

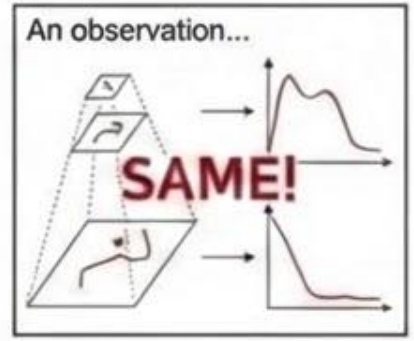
핵심 알고리즘: 특징점 검출 → 추적 → 궤적 평활화

① 특징점 검출 및 추적

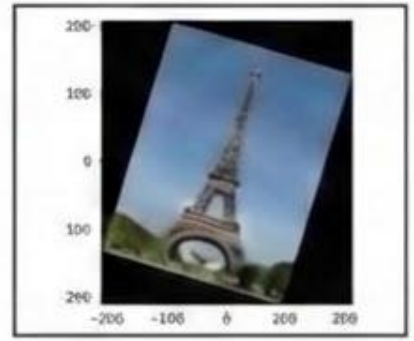
- Shi-Tomasi (goodFeaturesToTrack): 추적하기 유리한 코너점 50~200개 추출. 해리스 코너보다 연산 효율이 좋아 임베디드 장비에 적합



- LK Optical Flow (PyrLK): 피라미드 기법으로 UGV 주행 중 큰 폭의 진동부터 미세한 떨림까지 계층적으로 추적 → 움직임 벡터 산출

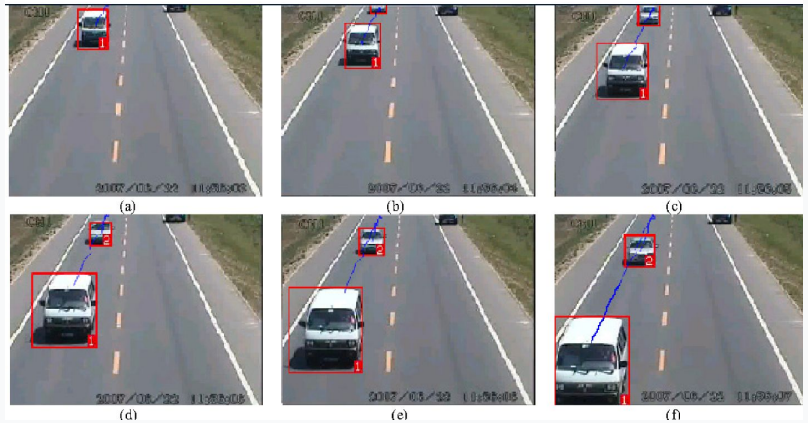


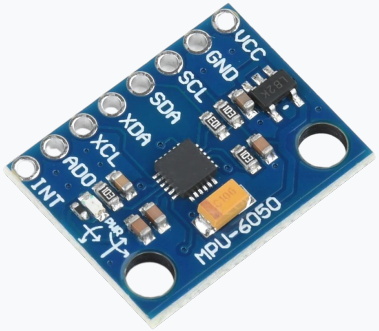
- 2D Affine Partial 행렬 추정: x, y, angle 3차원으로 흔들림 궤적 분해



비디오 안정화 - 알고리즘 구조

핵심 알고리즘: 특징점 검출 → 추적 → 궤적 평활화





② 센서 융합형 칼만 필터

(Sensor-Fused Kalman Filter)

- **MPU-6050 자이로 센서 연동:** 물리적 각속도를 실시간 판독
- **적응형 R값 제어 (Adaptive Control):**

```
r_theta_ = r_trans_x_ = r_trans_y_ = (abs(gyro_z) > 10.0 || abs(gyro_y) > 10.0) ? kRFast : kRStable;
```

💡 DIS의 고질적 약점인 "의도된 회전" vs "외부 충격 흔들림" 구분 불가 문제를 하드웨어로 해결

영상 밝기 보정 – 알고리즘 비교 및 선택

야외/역광 환경 대응: 3가지 기법 테스트

CLAHE

방식: BGR → LAB 변환
→ L채널 히스토그램 평활화

✗ **임베디드 부적합:**
매 프레임 변환/연산
→ CPU 극심한 부하

감마 보정 (Retinex)

방식: 각 픽셀 비선형 지수 연산 pow()

✗ **성능 이슈:**
GPU/CUDA 없이
15fps 이상 불가

✓ 선형 밝기 보정 (최종 선택)

방식: 선형 변환 (O(N))
(결과 = alpha * 원본 + beta)

✓ **최적합:** CPU 친화적, 실시간성 확보

결정 사유 (Decision Rationale)

화질 디테일은 CLAHE보다 낮으나, 실시간성이 필수인 UGV 도메인에서
'속도 대비 효과(가성비)'가 가장 우수하여 최종 채택함.

영상 밝기 보정 - 아키텍처 변화와 최종 판단

영상 밝기 보정 처리 위치 3번 변경

① 최초: VMS_Server

서버 CPU 연산 과부하로 인한 병목 우려 → 배제

② 중간: Gateway

디코딩 → 보정 → 재인코딩
이중 오버헤드 → 실시간 불가

③ 최종 설계: UGV_APP

Raw 프레임 보정 후 단 1번 인코딩
엣지 컴퓨팅으로 오버헤드 제거

최종 폐기(Rollback) 결정 사유

UGV_APP으로 최적화했음에도 연산 추가 시 프레임 끊김(Frame Drop) 현상 발생.

> **결론 : UGV 원격 조종에서 가장 중요한 것은 더 또렷한 화질이 아니라,
> 조이스틱 조작과 영상 간의 딜레이(Latency) 최소화 및 프레임 드롭 방지이다.**

→ **초저지연(Zero-latency) 파이프라인 확보를 1순위로 판단, 영상 밝기 보정 기능 최종 제거**

CCTV - 이상온도 바운딩 박스 구현

TNO-C3020TRA 열화상 카메라 연동

- 특정 온도 범위(-20°C ~ 129°C) 물체를 화면에 빨간색 박스로 표시
- **한계**: CCTV 자체에 '불(Fire)' 형상 판별 AI가 부재하여, 엔진 등 단순 고온 물체에도 박스를 렌더링함

제안

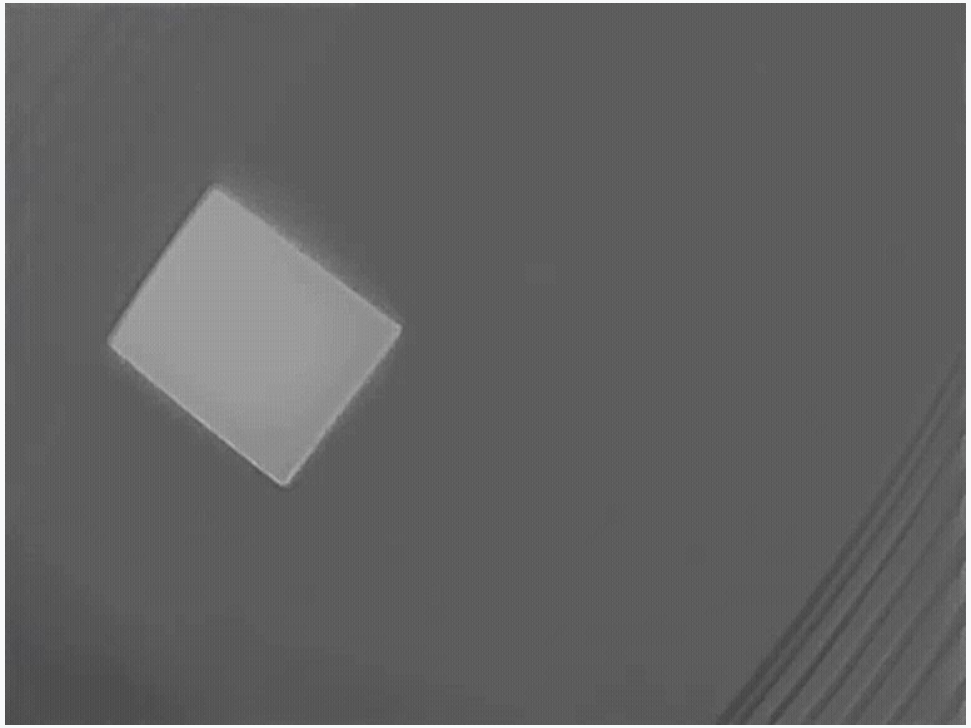
향후 비전 AI 기반 화재 판별 고도화 탑재 권장

바운딩박스 오버레이 파이프라인 (VMS_Server 단)

- **스트리밍 분기:** CCTV-TIC 타입 감지 → 일반 Proxy 대신 fireTracker로 분기
- **영상 디코딩 + 렌더링:** RTSP 수신 → cv::Mat 디코딩

→ 이상온도 ROI 좌표 맵핑 → cv::rectangle 오버레이

- **재인코딩 + 송출:** 가공된 프레임 H.264 인코딩 → 로컬 RTSP 송출 → VMS_Qt 실시간 수신

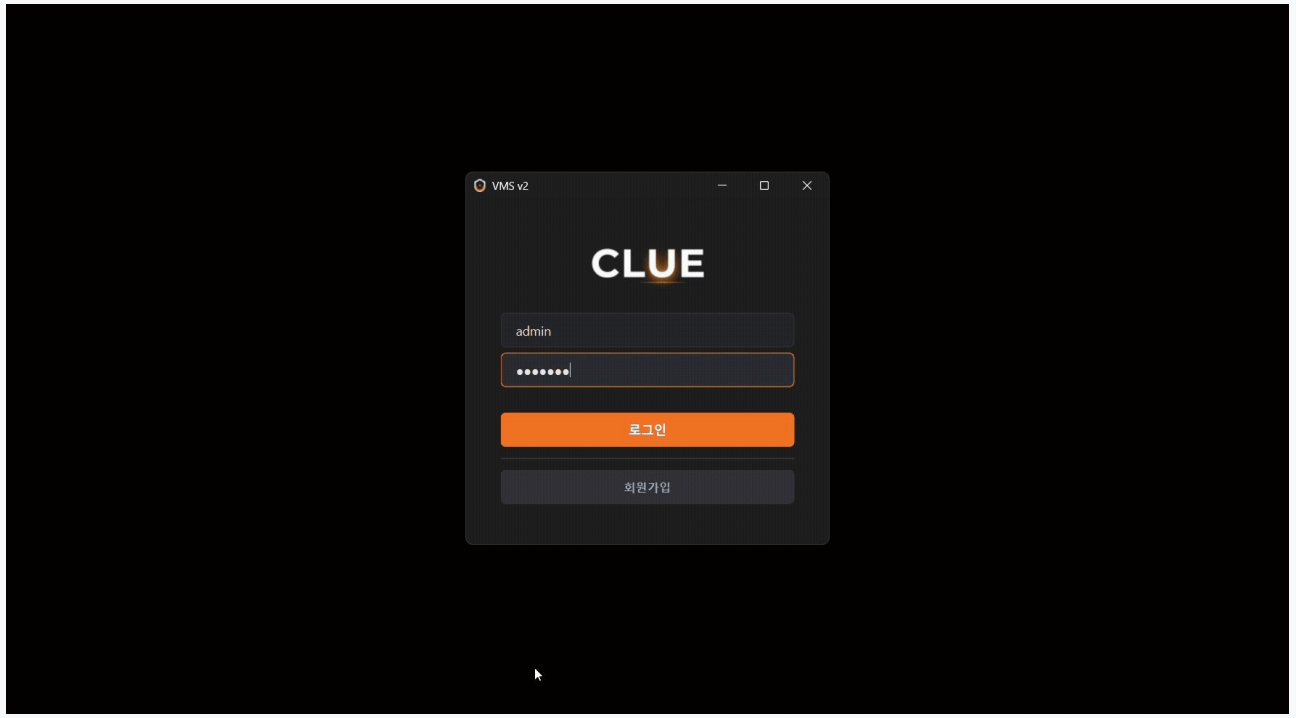


⚠ ONVIF 이벤트 중복 수신 이슈

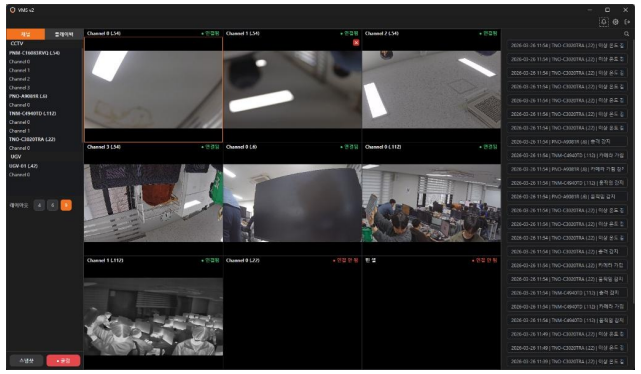
온도 임계치 초과 시 BoxTemperature와 Alarm 2개 이벤트가 동시 인입되어 DB 및 트래픽 낭비 발생.

VMS Client

김가영 | Qt VMS 통합 클라이언트

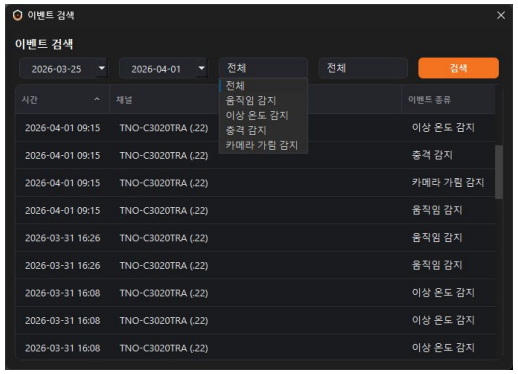


VMS: 주요 기능



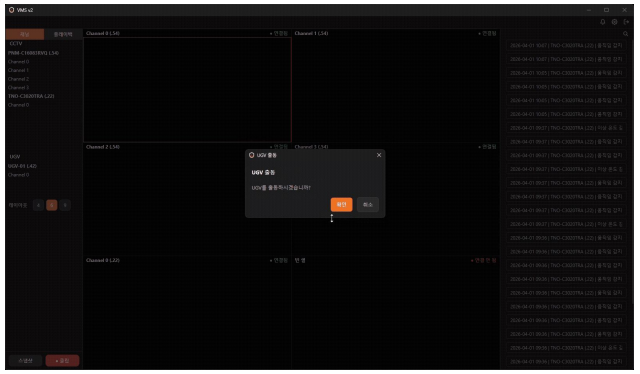
멀티뷰 관제

4 / 6 / 9 채널 분할 실시간 스트리밍



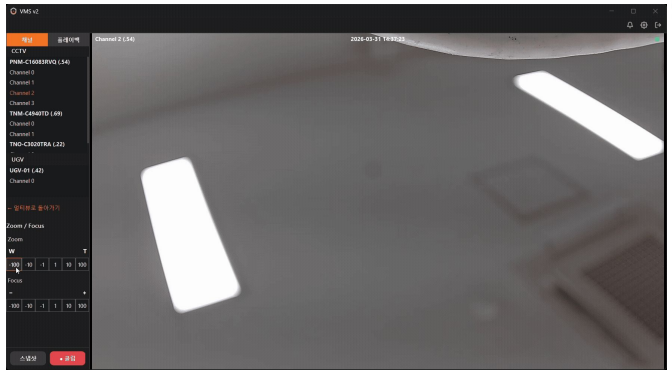
이벤트

WS 실시간 수신 + 검색 + 알림 센터



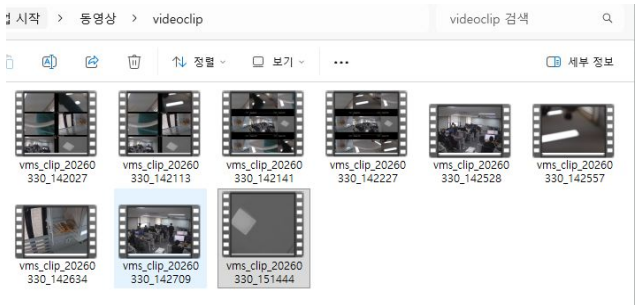
UGV 출동

이벤트 → 출동 → UGV 전체화면 제어



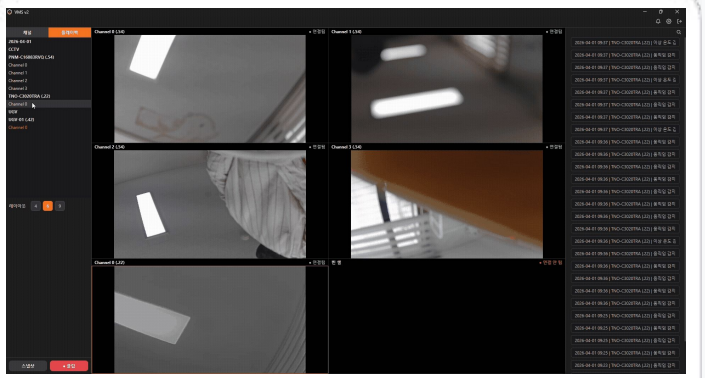
CCTV 제어

전체화면 줌 / 포커스 (서버 API 경유)



스냅샷 / 클립

설정 경로에 PNG / MP4 저장



Playback

타임라인 → 구간 클릭 재생

VMS: 아키텍처

UI Layer

MainScreen · CctvScreen · UgvScreen · Playback · Login

Service Layer

AuthService · DeviceService · EventService · UgvService

Infra Layer

RestClient (HTTP/JWT) · WsClient (WS)

Media Layer

ChannelSessionManager · StreamPlayer · VideoRenderWidget

핵심 설계 포인트

- **단일 책임:** UI -> Service -> Infra 단방향 호출
- **변경 영향 최소화:** 화면 API 변경 시 Service 계층만 수정
- **인프라 계층 분리:** JWT·REST·WS를 UI/서비스 계층과 분리
- **미디어 계층 분리:** 스트림 세션이 UI와 독립적으로 동작

VMS: version 1에서 드러난 구조적 한계

⚠ 렌더링 엔진

OSD 불안정, 검은 화면 캡처

d3d11이 윈도우에 네이티브 렌더링

⚠ 클립 저장

10초 클립에 35~469초

PNG 직렬 저장 → ffmpeg 재읽기 → 디스크 I/O 왕복

⚠ 멀티뷰 성능

FPS 0.2~17.6 불안정

모든 채널 최고 품질 + hidden 상태에서도 dispatch

한계 영역	구체적 증상	발생 원인
Playback 결합	타임라인 수정 시 연쇄 장애 발생	timeline / marker / 재생 URL 간의 강한 결합도
UGV 세션 모호	명령 1개 실패 시 전체 UGV 제어 세션이 Error 처리됨	ACK / error / telemetry / 제어가 한 흐름에 얽혀 있음
알림 혼재	동일한 오류가 어디선 팝업, 어디선 상태라벨	공통 정책 없이 각 화면(UI)에서 개별적으로 구현

StreamQualityProfile 3단계
(평균 3.15초)
+ 화면에 보이는 채널만 렌더링 (FPS 5.5~7.7 안정)

VMS: v1 → v2 핵심 아키텍처 개선 및 대응

렌더링 엔진

v1 **d3d11videosink**
(Windows 창 직접 렌더, 네이티브 윈도우)

↓

v2 **appsink + QOpenGLWidget**
OSD 안정화 / 캡처 소스 통일

화면 캡처

v1 **BitBlt**
(검은 화면 현상)

↓

v2 **grabFramebuffer()**
OpenGL FBO 직접 읽기 (정확한 픽셀 보장)

클립 저장

v1 **프레임 → PNG 저장 → ffmpeg 재읽기**
(35~469초 소요)

↓

v2 **rawvideo → ffmpeg stdin pipe**
(평균 3.15초)

멀티뷰 성능

v1 **모든 채널 풀 품질 + 숨겨진 채널도 렌더링**
(FPS 0.2~17.6 불안정)

↓

v2 **StreamQualityProfile 3단계**
+ 화면에 보이는 채널만 렌더링
(FPS 5.5~7.7 안정)

VMS: 트러블슈팅 #1: 클립 저장 병목 현상

[측정 조건] 10초 분량 클립 반복 측정 (멀티뷰 vs 전체 화면)

분석 지표	Before (version 1)	After (version 2)
데이터 처리 경로	프레임 → PNG 파일 직렬화 → ffmpeg 재읽기	프레임 → rawvideo → ffmpeg stdin pipe
디스크 I/O 횟수	저장할 전체 프레임 수 × 2회	MP4 최종 파일 1회 기록
주요 성능 병목점	PNG 압축 인코딩 비용 + 재해제(디코딩) 비용	불필요한 압축 과정 완벽 제거
멀티뷰 (10초)	평균 70초 (35 ~ 154초)	평균 3.15초 (0.7 ~ 5.4초) -> 93% 감소
전체 화면 (10초)	평균 409초 (339 ~ 469초)	평균 6.0초 (3.7 ~ 9.2초) -> 98% 감소

수정 포인트:
캡처 프레임을 rawvideo 바이트 스트림으로 stdin에 직접 전달

결과:
비동기 백그라운드 처리로 UI 완전 멈춤(블로킹) 현상 근절

VMS: 트러블슈팅 #2 - 화면 전환 지연

원인 분석 및 해결 방안

- **추적:** UI 전환 함수는 빠르게 동작하는데 체감이 느림

[측정 조건] Debug x64, 동일 장비·동일 네트워크 반복 측정

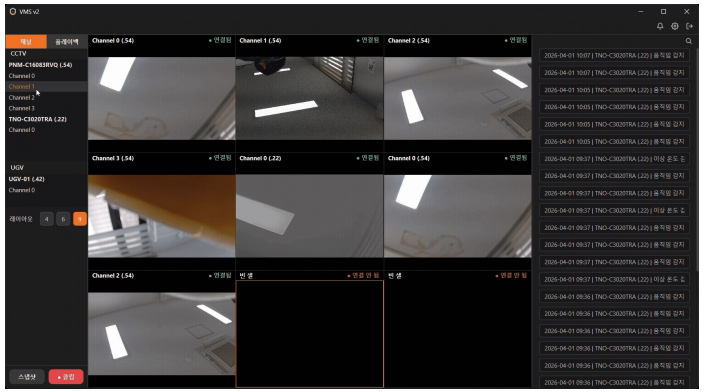
> **screen_transition_ms**
> UI 함수 실행 시간: 20~76ms (정상/빠름)
>
> **first_live_frame_after_transition_ms**
> 실제 라이브 영상 도달: **~2,100ms (병목 구간)**

근본 원인 확인:

멀티뷰(DenseGrid)에서 단일채널(Normal) 프로파일 전환 시 발생하는 **pipeline 전체 재시작**이 병목 원인

측정 결과 및 지표 분리

- **개선 조치:** GStreamer **capsfilter caps** 값만 런타임에 교체하여 파이프라인 유지 정책 적용



체감 지연 시간 전후 비교:

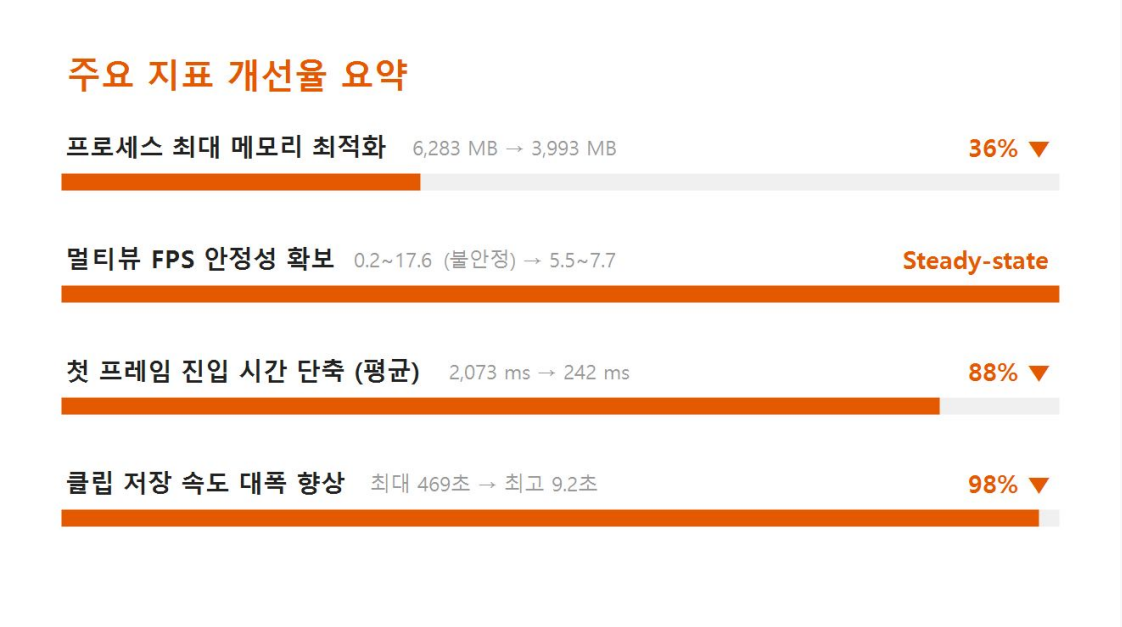
1,989 ~ 2,158 ms

→ 175 ~ 304 ms

VMS: 성능 결과 요약

핵심 성능 최적화 결과

- **프로세스 최대 메모리 최적화**
  6,283 MB → 3,993 MB **(-2,290 MB 절감)**
- **멀티뷰 FPS 안정성 확보**
  0.2~17.6 (불안정)
  → **5.5~7.7 (Steady-state 유지)**
- **첫 프레임 진입 시간 단축 (평균)**
  2,073 ms → 242 ms **(지연율 88% 감소)**
- **클립 저장 속도 대폭 향상 (전체화면 10초 클립)**
  최대 469초
  → 최고 9.2초 **(소요시간 98% 감소)**



Next Step

CPU 디코딩을 GPU 하드웨어 디코딩으로 전환해 멀티뷰 성능의 근본 병목을 해소할 예정

Q&A

이창호

팀장 /
Gateway & VMS Server

김가영

VMS Client

김태훈

영상처리

한정수

UGV BSP & Application

감사합니다.

팀명: Clue

github: https://github.com/VEDA-3-Clue

Google Drive:
https://drive.google.com/drive/folders/15C-3YntFYA9n76opzS8_Ii2Uk_18nr2v